

Table 1. Comparison between the Proposed PHBF and Related Works

| References           | Low Storage | Fast Query | Protection of Enrolled Data | In-house Noise Handling Property | Low Cost |
|----------------------|-------------|------------|-----------------------------|----------------------------------|----------|
| [19]                 | ×           | ×          | ✓                           | ×                                | ×        |
| [10]                 | ×           | ×          | ✓                           | ×                                | ×        |
| [16]                 | ×           | ×          | ✓                           | ×                                | ×        |
| [18]                 | ×           | ×          | ✓                           | ×                                | ×        |
| [9]                  | ×           | ×          | ✓                           | ×                                | ×        |
| [2]                  | ×           | ×          | ✓                           | ×                                | ×        |
| <b>Proposed PHBF</b> | ✓           | ✓          | ✓                           | ✓                                | ✓        |

the PUF response that may cause authentication to fail. Even though the PUF error can be mitigated using ECC within the chip, it would introduce significant overhead and create the possibility of CRP leakage through side-channel attack [1]. Work in [2] is also based on developing a counterfeit mitigation system which runs inside a blockchain using a smart contract. Similar to [18], this system is not noise-tolerant. Furthermore, the framework does not track location information across the supply chain. A blockchain-based verification system that can store transaction time, chip marking, and IDs is proposed in [19]. Even though their solution can detect overproduced, remarked, and recycled ICs, scalability is a significant concern in the blockchain-based framework. Existing studies present various fast, storage-efficient, and error-tolerant data structures for biometric authentication. For example, Shomaji et al. proposed a novel probabilistic data structure, HBF, that can handle noise in data and provides fast and storage-efficient authentication [17]. However, there was no extended work to leverage HBF for IC authentication, and incorporating timestamps in HBF is still an open research question. Table 1 compares the proposed PHBF with relevant works with respect to storage requirement, query speed, noise tolerance, the privacy of the enrolled template, robustness, and computational cost. The term, “In-house” means that inherently, by the nature of PHBF, the noise can be handled during query. It is within the process of query processing step, and we do not need a separate module to achieve this. Moreover, In a supply chain, myriads of challenges may arise, such as increased time range, storage requirements, changes in location, or even ICs altering their PUF responses due to noise. Our proposed method, PHBF, can sustain these changes and continue to perform optimally. It is designed to handle variations in these parameters without causing inconsistencies in performance which makes it robust.”

### 3 Background

In this section, we introduce some preliminary concepts that are related to our proposed system. Multiple notations have been used in this article which is described in Table 2.

**BF.** A BF is a probabilistic data structure that is used to test whether an element is a member of a set [3]. The idea behind a BF is to use a fixed-size bit array and a set of hash functions to map elements to positions in the array. When an element is added to the set, the corresponding bits in the array are set to “1.” When a query is made to check if an element is in the set, the corresponding bits are checked and if all of them are “1,” the element is considered to be in the set (see Figure 1(a)). However, if any of the bits are “0,” the element is definitely not in the set. FP rate of a BF is the probability that a membership query for an item returns “probably in set” when the item is actually not in the set. The FP rate is a crucial metric for BF, as it determines the accuracy of the filter in identifying membership in a set. If  $m$  number of bits are used to build a BF and  $k$  number of hash functions are used on each instance of the  $n$  number of elements, the probability of FP is,  $Prob(p) = (1 - e^{-k(n/m)})^k$ . However, BFs are inadequate in handling corrupt or noisy data. While doing a string matching operation, if the query string is almost as similar as the enrolled string but not exactly a duplicate; while querying, it will return a negative result in the membership testing. Thus, one small change in the query string may result in different hash values.

indicate that the PHBF emerges as the most temporally efficient solution among the considered alternatives.

*Time Requirement for Proposed Query Sets.* To detect theft counterfeits, our proposed method comprises the querying into every location (see Figure 6). Quite similar to the theft counterfeit, if the IC cannot be found at the OEM-end, to detect the cloned and overproduced ICs (see Figure 7), query needs to be performed in remaining location PHBFs as well. Thus, the query complexity to detect theft, cloned and overproduced IC can be written as,  $QueryTime = L \cdot (\log_2(T))$ .

The time complexity of detecting recycled ICs (see Figure 8) is same as query time complexity in an HBF. We calculate the query time using, number of BFs in an HBF  $\times$  number of hash functions. Mathematically, the equation can be represented as,  $QueryTime = N \cdot K$ .

## 7 Conclusion and Future Work

We have presented a novel data structure PHBF which is a fast, storage-efficient and noise-tolerant framework to offer a scalable and robust authentication and tracking of ICs. We discussed the design principles, the flow of IC enrollment and query process for detecting four major types of counterfeits. We also analyzed the authentication performance and scalability in terms of storage and time efficiency in both theoretically and experimentally. Furthermore, we compared our results with four existing state-of-the-art approaches and observed that our proposed PHBF performs significantly better in terms of authentication performance and scalability properties.

In our future work, we will focus on several key areas to further enhance the PHBF framework. These include the integration of a deletion and insertion property within the PHBF structure, allowing for more flexible and dynamic data management by dynamically adding or removing items. This will enable our proposed framework to be a Dynamic PHBF framework. Furthermore, we intend to do a thorough security analysis to assess the framework's resilience against potential adversarial models which will ensure that it can withstand various types of attacks. Finally, we will explore improvements to the overall framework to address any identified security or privacy issues.

## References

- [1] Abdulrahman Alaql, Tamzidul Hoque, Domenic Forte, and Swarup Bhunia. 2019. Quality obfuscation for error-tolerant and adaptive hardware IP protection. In *Proceedings of the 2019 IEEE 37th VLSI Test Symposium (VTS)*. IEEE, 1–6.
- [2] Leonardo Aniello, Basel Halak, Peter Chai, Riddhi Dhall, Mircea Mihalea, and Adrian Wilczynski. 2021. Anti-BIUFF: Towards counterfeit mitigation in IC supply chains using blockchain and PUF. *International Journal of Information Security* 20, 3 (2021), 445–460.
- [3] Burton H. Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM* 13, 7 (1970), 422–426.
- [4] Nathan G. Bronson, Jared Casper, Hassan Chafi, and Kunle Olukotun. 2010. A practical concurrent binary search tree. *ACM SIGPLAN Notices* 45, 5 (2010), 257–268.
- [5] Wenjie Che, Fareena Saqib, and Jim Plusquellic. 2015. PUF-based authentication. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD '15)*. IEEE Press, 337–344.
- [6] Yansong Gao, Said F. Al-Sarawi, and Derek Abbott. 2020. Physical unclonable functions. *Nature Electronics* 3, 2 (2020), 81–91.
- [7] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity search in high dimensions via hashing. In *Proceedings of the 25th VLDB Conference*, Vol. 99, 518–529.
- [8] Jan L. Harrington. 2016. *Relational Database Design and Implementation*. Morgan Kaufmann, Cambridge, MA.
- [9] Md Nazmul Islam and Sandip Kundu. 2019. Enabling IC traceability via blockchain pegged to embedded PUF. *ACM Transactions on Design Automation of Electronic Systems* 24, 3 (2019), 1–23.
- [10] Md Nazmul Islam, Vinay C. Patii, and Sandip Kundu. 2018. On IC traceability via blockchain. In *Proceedings of the 2018 International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*. IEEE, 1–4.
- [11] Abhranil Maiti and Patrick Schaumont. 2011. Improved ring oscillator PUF: An FPGA-friendly secure primitive. *Journal of Cryptology* 24 (2011), 375–397.
- [12] Judy Meiksin. 2022. US “CHIPS and science act” gives semiconductor R&D and industry a multibillion dollar boost. *MRS Bulletin* 47 (2022), 890–892.

- [13] Michael Nofer, Peter Gomber, Oliver Hinz, and Dirk Schiereck. 2017. Blockchain. *Business & Information Systems Engineering* 59, 3 (2017), 183–187.
- [14] Landon Curt Noll. 1994. FNV HASH. Retrieved February 23, 2023 from <http://www.isthe.com/chongo/tech/comp/fnv/>
- [15] Yanqing Peng, Jinwei Guo, Feifei Li, Weining Qian, and Aoying Zhou. 2018. Persistent bloom filter: Membership testing for the entire history. In *Proceedings of the 2018 International Conference on Management of Data*. ACM, 1037–1052.
- [16] S. S. Rekha, K. Suraj, and K. Sudeendra Kumar. 2021. A holistic blockchain based IC traceability technique. In *Proceedings of the 2021 IEEE International Symposium on Smart Electronic Systems (iSES) (Formerly iNiS)*. IEEE, 307–310.
- [17] Sumaiya Shomaji, Fatemeh Ganji, Damon Woodard, and Domenic Forte. 2019. Hierarchical bloom filter framework for security, space-efficiency, and rapid query handling in biometric systems. In *Proceedings of the 2019 IEEE 10th International Conference on Biometrics Theory, Applications and Systems (BTAS)*. IEEE, 1–8.
- [18] Nidish Vashistha, Muhammad Monir Hossain, Md Rakib Shahriar, Farimah Farahmandi, Fahim Rahman, and Mark M. Tehranipoor. 2021. eChain: A Blockchain-Enabled ecosystem for electronic device authenticity verification. *IEEE Transactions on Consumer Electronics* 68, 1 (2021), 23–37.
- [19] Xiaolin Xu, Fahim Rahman, Bicky Shakya, Apostol Vassilev, Domenic Forte, and Mark Tehranipoor. 2019. Electronics supply chain integrity enabled by blockchain. *ACM Transactions on Design Automation of Electronic Systems* 24, 3 (2019), 1–25.

Received 11 November 2023; revised 30 August 2024; accepted 8 July 2025

Table 2. Classification Scores for Detecting Program Instruction

| Device     | Classifier [26]         | Validation Score | F1-Score | Recall | Specificity | Precision | Accuracy |
|------------|-------------------------|------------------|----------|--------|-------------|-----------|----------|
| FPGA       | SVM (Kernel: Linear)    | 92.8%            | 92.6%    | 92.7%  | 99.3%       | 92.7%     | 92.6%    |
|            | AdaBoost                | 91.8%            | 92.0%    | 92.0%  | 99.3%       | 92.1%     | 92.0%    |
|            | Linear Discriminant     | 86.6%            | 87.2%    | 87.3%  | 98.8%       | 87.3%     | 87.3%    |
| ATmega328P | SVM (Kernel: Quadratic) | 95.0%            | 96.1%    | 96.1%  | 99.6%       | 96.1%     | 96.1%    |
|            | Bagged trees            | 92.5%            | 91.6%    | 91.7%  | 99.2%       | 91.6%     | 91.6%    |
|            | Linear Discriminant     | 88.9%            | 92.2%    | 92.0%  | 99.2%       | 92.8%     | 92.2%    |

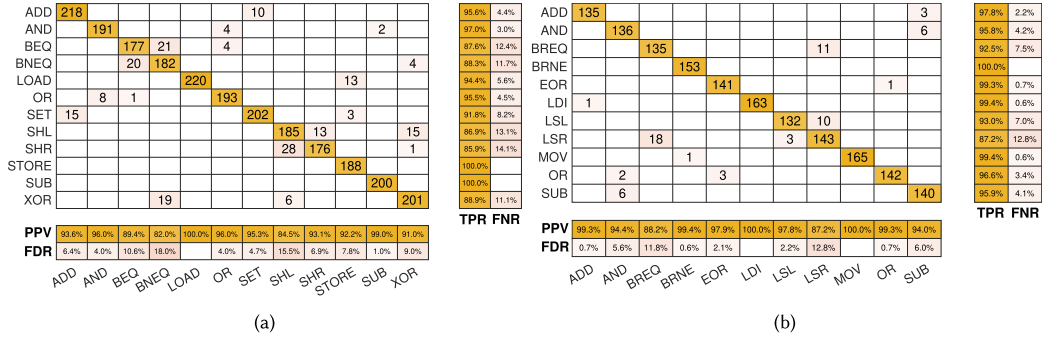


Fig. 8. Confusion matrix of reverse-engineered program instructions: (a) Artix 7 FPGA, (b) ATmega328P microcontroller.

detect and classify program instructions. Each model is trained using 10-fold cross-validation to enhance stability and prevent overfitting. The best-performing classifier is selected based on the average cross-validation accuracy.

After extracting instruction-specific features from the impedance traces as outlined in previous sections (Sections 5.2 and 5.3), the models are evaluated on a 30% held-out test set. The SVM performs exceptionally well and achieves top classification performance, with F1-score, precision, recall, specificity, and accuracy all over 92% on the test data. This demonstrates the viability of our proposed approach using impedance side-channel for reverse-engineering program instructions. The findings of the classification of individual program instructions are presented in Table 2. The confusion matrix in Figure 8 provides further insight into the performance of the top classifier, SVM, along with the **positive predictive value (PPV)**, **false-discovery rate (FDR)**, TPR, and false-negative rate. PPV represents the probability that a predicted result is actually correct, whereas FDR is the complement of PPV. A high PPV indicates the reliability of the trained model, and the false positive rate is the complement of specificity. Among the other classifiers, SVM demonstrates the ability to accurately differentiate between the various instructions with minimal confusion and a top 10-fold cross-validation score.

In summary, by leveraging ML algorithms and extracted signal features, our method of using impedance as a side-channel can reliably identify and reverse-engineer program instructions executed on a chip. The exceptional classification results from the SVM highlight the promise of this technique. While our ML pipeline-comprising PCA for feature extraction and standard classifiers such as SVM, kNN, and AdaBoost-is not novel, its application to impedance-based side-channel signals is unexplored. Existing studies primarily rely on power analysis or EM emissions.

In contrast, we demonstrate that even well-established ML models can extract meaningful instruction-specific patterns from impedance data. This practical demonstration validates impedance as a viable side-channel modality.

## 6 Related Works

*SCA in Hardware Defense:* Side channels in hardware have been leveraged both for extracting sensitive information and securing systems. For example, the authors in [25] use timing-based information leakage and simple power analysis to analyze vulnerabilities in embedded neural networks. In [19], an EM side-channel-based hardware Trojan detection is proposed by spectrum modeling and analyzing. Impedance can be employed as a tool for detecting malicious system as well as other physical side-channel attacks, e.g., power side-channel attacks. Authors in [47] detect board-level modifications by measuring impedance at various locations. Another approach in [17] detects Trojans by monitoring impedance changes in integrated circuit wires. Authors in [29] present a detection system that monitors battery impedance to identify malicious probing attempts in power side-channel attacks. By detecting impedance changes, their approach provides a new way to thwart such attacks through impedance-based sensing. Thus, prior works have demonstrated side channels like timing, power, EM emissions, and impedance can be exploited for analyzing hardware vulnerabilities, detecting Trojans, and identifying malicious modifications. Our research similarly uses impedance as a side-channel, but with the novel goal of reverse-engineering software instructions.

*SCA in Software Defense:* Recent work has leveraged physical side channels produced by embedded microcontrollers to infer internal behavior and operations. For example, authors in [10] present an analytical approach to assess the security of **hardware performance counters (HPCs)** for malware detection. Their framework analyzes the probability of malware detection given a program, set of HPCs, and sampling rate. In [13], the authors present a method utilizing power side-channel and ML on statistical trace features to detect runtime malware in medical devices. Analyzing EM emissions is another approach explored in prior art [18] detected malicious **programmable logic controller (PLC)** code by training a sequential neural network model on pre-processed frequency data to encode control flow transitions. Other works include detecting behavioral changes in industrial control systems [40] and identifying malicious PLC code [11]. Beyond EM and power analysis, recent work has investigated backscattering side channels [30]. In these attacks, adversaries analyze subtle variations in the reflected signal, which are modulated by impedance changes within the device.

*SCA in Code Reverse Engineering:* The authors in [16] implement an instruction classifier for the PIC16F687 microcontroller by performing a template attack using the power side-channel. With statistical models like **hidden Markov models (HMMs)**, they achieve 70.1% accuracy over 35 test instructions. By modeling control flows with HMMs on an 8-bit AVR, the authors in [24] demonstrate how voltage drops on the power pin can be measured to track code execution and detect anomalies. Similarly, the authors in [28] utilize instruction-level power side-channel leakage profiles to identify program instructions executed on an embedded processor. Other works include the use of localized EM probes to capture high-resolution side-channel traces for dynamic code recognition, as demonstrated in [39].

The extensive prior art demonstrates side channels can reveal rich details about a device's internal operations. Our work similarly leverages impedance as a side-channel, but with the unique goal of reverse-engineering software instructions on FPGA and microcontroller platforms. By contrast, most existing techniques focus on malware detection, behavior identification, and code profiling rather than reverse engineering the actual opcodes. The ability to reliably reconstruct sequences

of software instructions will open up new possibilities for observation, verification, and reverse engineering of embedded and cyber-physical systems.

## 7 Conclusion

In this article, we investigated the feasibility of using impedance as a side-channel signal to identify program instructions. Experimental validation was performed on two platforms with distinct instruction sets, an Artix-7 FPGA and an ATmega328P microcontroller, demonstrating instruction classification accuracies of 92.6% and 96.1%, respectively. By analyzing impedance variations induced by instruction-level switching activity, we extracted discriminative signal patterns across selected frequency channels and applied classical ML models to achieve high-accuracy classification. While the classifiers used are well-established, their application to impedance-based side channels is novel, revealing a previously unexplored attack surface. To support our findings, we developed a refined two-stage frequency selection method and introduced a gate-level mathematical formulation linking switching activity to impedance behavior. These results demonstrate that runtime impedance can effectively reflect executed instructions with high accuracy, opening new possibilities for code recognition, control flow analysis, reverse engineering, and embedded systems security. Importantly, the underlying leakage arises from fundamental switching behavior in digital logic gates and is therefore expected to generalize across architectures. Although this study focused on low-complexity platforms, future work will extend the methodology to more advanced architectures, including ARM Cortex-M and RISC-V. Such extensions present additional challenges, such as instruction overlap, increased background activity, and limited PDN access. Addressing these complexities may require advanced triggering, signal processing, or on-chip impedance sensing techniques. By establishing impedance as an active side-channel modality rather than a passive electrical property, this work introduces a new perspective on system observability and highlights the importance of exploring unconventional leakage sources to enhance cyber resilience in increasingly interconnected and security critical computing environments.

## Acknowledgments

We would like to thank the reviewers for providing their valuable feedback.

## References

- [1] Alchitry. 2024. Alchitry Au. Retrieved September 9, 2024 from <https://alchitry.com/boards/au>
- [2] Arduino UNO. 2024. Arduino Uno Rev3. Retrieved September 9, 2024 from <https://docs.arduino.cc/hardware/uno-rev3>
- [3] ATmega328P Specification and Datasheet. 2024. ATmega328P Specification and Datasheet. Retrieved September 9, 2024 from [https://www1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P\\_Datasheet.pdf](https://www1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf)
- [4] Md Sadik Awal, Buddhipriya Gayanath, and Md Tauhidur Rahman. 2024. Impedance vs power side-channel vulnerabilities: A comparative study. arXiv:2405.06242. Retrieved from <https://arxiv.org/abs/2405.06242>
- [5] Md Sadik Awal, Arjuna Madanayake, and Md Tauhidur Rahman. 2022. Nearfield RF sensing for feature-detection and algorithmic classification of tamper attacks. *IEEE Journal of Radio Frequency Identification* 6 (2022), 490–499.
- [6] Md Sadik Awal and Md Tauhidur Rahman. 2023. Disassembling software instruction types through impedance side-channel analysis. In *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 227–237. DOI: 10.1109/HOST55118.2023.10133318
- [7] Md Sadik Awal and Md Tauhidur Rahman. 2025. DEXim: Exposing impedance-based data leakage in emerging memories. In *2025 58th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE.
- [8] Md Sadik Awal, Christopher Thompson, and Md Tauhidur Rahman. 2022. Utilization of impedance disparity incurred from switching activities to monitor and characterize firmware activities. In *2022 IEEE Physical Assurance and Inspection of Electronics (PAINE)*. IEEE, 1–7.
- [9] R. Jacob Baker. 2019. *CMOS: Circuit Design, Layout, and Simulation*. John Wiley & Sons.
- [10] Kanad Basu, Prashanth Krishnamurthy, Farshad Khorrami, and Ramesh Karri. 2019. A theoretical study of hardware performance counters-based malware detection. *IEEE Transactions on Information Forensics and Security* 15 (2019), 512–525.

- [11] Nathaniel Boggs, Jimmy C. Chau, and Ang Cui. 2018. Utilizing electromagnetic emanations for out-of-band detection of unknown attack code in a programmable logic controller. In *Cyber Sensing 2018*, Vol. 10630, SPIE, 84–99.
- [12] Martin Brisfors, Michail Moraitis, and Elena Dubrova. 2022. Side-channel attack countermeasures based on clock randomization have a fundamental flaw. *Cryptology ePrint Archive*. Retrieved from <https://eprint.iacr.org/2022/1416>
- [13] Shane S. Clark, Benjamin Ransford, Amir Rahmati, Shane Guineau, Jacob Sorber, Wenyuan Xu, and Kevin Fu. 2013. {WattsUpDoc} Power side channels to nonintrusively discover untargeted malware on embedded medical devices. In *2013 USENIX Workshop on Health Information Technologies (HealthTech '13)*.
- [14] Stephen Dunlap, Jonathan Butts, Juan Lopez, Mason Rice, and Barry Mullins. 2016. Using timing-based side channels for anomaly detection in industrial control systems. *International Journal of Critical Infrastructure Protection* 15 (2016), 12–26.
- [15] Joel P. Dunsmore. 2020. *Handbook of Microwave Component Measurements: With Advanced VNA Techniques*. John Wiley & Sons.
- [16] Thomas Eisenbarth, Christof Paar, and Björn Weghenkel. 2010. Building a side channel based disassembler. In *Transactions on Computational Science X: Special Issue on Security in Computing*, Part I. Springer, 78–99.
- [17] Daisuke Fujimoto, Shota Nin, Yu-Ichi Hayashi, Noriyuki Miura, Makoto Nagata, and Tsutomu Matsumoto. 2018. A demonstration of a HT-detection method based on impedance measurements of the wiring around ICs. *IEEE Transactions on Circuits and Systems II: Express Briefs* 65, 10 (2018), 1320–1324.
- [18] Yi Han, Sriharsha Etigowni, Hua Liu, Saman Zonouz, and Athina Petropulu. 2017. Watch me, but don't touch me! Contactless control flow monitoring via electromagnetic emanations. In *2017 ACM SIGSAC Conference on Computer and Communications Security*, 1095–1108.
- [19] Jiaji He, Yiqiang Zhao, Xiaolong Guo, and Yier Jin. 2017. Hardware trojan detection through chip-free electromagnetic side-channel statistical analysis. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 10 (2017), 2939–2948.
- [20] Kengo Iokibe, Tomonobu Kan, and Yoshitaka Toyota. 2020. A study on evaluation board requirements for assessing vulnerability of cryptographic modules to side-channel attacks. In *2020 IEEE International Symposium on Electromagnetic Compatibility & Signal/Power Integrity (EMCSI)*. IEEE, 528–531.
- [21] J. Edward Jackson. 2005. *A User's Guide to Principal Components*. John Wiley & Sons.
- [22] Jingfei Kong, Onur Aciicmez, Jean-Pierre Seifert, and Huiyang Zhou. 2009. Hardware-software integrated approaches to defend against software cache-based side channel attacks. In *2009 IEEE 15th International Symposium on High Performance Computer Architecture*. IEEE, 393–404.
- [23] Lang Lin, Markus Kasper, Tim Güneysu, Christof Paar, and Wayne Burleson. 2009. Trojan side-channels: Lightweight hardware trojans through side-channel engineering. In *11th International Workshop on Cryptographic Hardware and Embedded Systems (CHES '09)*. Springer, 382–395.
- [24] Yannan Liu, Lingxiao Wei, Zhe Zhou, Kehuan Zhang, Wenyuan Xu, and Qiang Xu. 2016. On code execution tracking via power side-channel. In *2016 ACM SIGSAC Conference on Computer and Communications Security*, 1019–1031.
- [25] Saurav Maji, Utsav Banerjee, and Anantha P. Chandrakasan. 2021. Leaky nets: Recovering embedded neural network models and inputs through simple power and timing side-channels—attacks and defenses. *IEEE Internet of Things Journal* 8, 15 (2021), 12079–12092.
- [26] Stephen Marsland. 2011. *Machine Learning: An Algorithmic Perspective*. Chapman and Hall/CRC.
- [27] Saleh Khalaj Monfared, Tahoura Mosavirik, and Shahin Tajik. 2023. LeakyOhm: Secret bits extraction using impedance analysis. In *2023 ACM SIGSAC Conference on Computer and Communications Security*, 1675–1689.
- [28] Mehari Msgna, Konstantinos Markantonakis, and Keith Mayes. 2014. Precise instruction-level side channel profiling of embedded processors. In *10th International Conference on Information Security Practice and Experience (ISPEC '14)*. Springer, 129–143.
- [29] Rowshon Munny and John Hu. 2021. Power side-channel attack detection through battery impedance monitoring. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [30] Luong N. Nguyen, Chia-Lin Cheng, Milos Prvulovic, and Alenka Zajić. 2019. Creating a backscattering side channel to enable detection of dormant hardware trojans. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 7 (2019), 1561–1574.
- [31] Luong N. Nguyen, Chia-Lin Cheng, Frank T. Werner, Milos Prvulovic, and Alenka Zajic. 2020. A comparison of backscattering, EM, and power side-channels and their performance in detecting software and hardware intrusions. *Journal of Hardware and Systems Security* 4 (2020), 150–165.
- [32] Luong N. Nguyen, Baki Berkay Yilmaz, Milos Prvulovic, and Alenka Zajic. 2020. A novel golden-chip-free clustering technique using backscattering side channel for hardware trojan detection. In *2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 1–12.
- [33] Marwan Omar. 2022. *Defending Cyber Systems through Reverse Engineering of Criminal Malware*. Springer Nature.

- [34] Max Panoff, Honggang Yu, Haoqi Shan, and Yier Jin. 2022. A review and comparison of AI-enhanced side channel analysis. *ACM Journal on Emerging Technologies in Computing Systems* 18, 3 (2022), 1–20.
- [35] J. M. Rabaey, A. P. Chandrakasan, and B. Nikolic. 2002. *Digital Integrated Circuits*. Prentice hall.
- [36] A. S. Smith and K. C. Smith. 2004. *Microelectronic Circuits* (5th. ed.). Oxford University Press.
- [37] Devin Spatz, Devin Smarra, and Igor Ternovskiy. 2019. A review of anomaly detection techniques leveraging side-channel emissions. *Cyber Sensing 2019* 11011 (2019), 48–55.
- [38] François-Xavier Standaert. 2010. Introduction to side-channel attacks. In *Secure Integrated Circuits and Systems*. Springer, 27–42.
- [39] Daehyun Strobel, Florian Bache, David Oswald, Falk Schellenberg, and Christof Paar. 2015. Scandalee: A side-channel-based disassembler using local electromagnetic emanations. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 139–144.
- [40] Pol Van Aubel, Kostas Papagiannopoulos, Łukasz Chmielewski, and Christian Doerr. 2018. Side-channel based intrusion detection for industrial control systems. In *12th International Conference on Critical Information Infrastructures Security (CRITIS '17)*, Vol. 12, Springer, 207–224.
- [41] Zhenting Wang, Kai Mei, Hailun Ding, Juan Zhai, and Shiqing Ma. 2022. Rethinking the reverse-engineering of trojan triggers. In *Advances in Neural Information Processing Systems*, Vol. 35, 9738–9753.
- [42] W. Wolf. 2004. *FPGA-Based System Design*. Pearson Education.
- [43] Jean-Paul A. Yaacoub, Ola Salman, Hassan N. Noura, Nesrine Kaaniche, Ali Chehab, and Mohamad Malli. 2020. Cyber-physical systems security: Limitations, issues and future trends. *Microprocessors and Microsystems* 77 (2020), 103201.
- [44] Yuan Yao, Mo Yang, Conor Patrick, Bilgiday Yuce, and Patrick Schaumont. 2018. Fault-assisted side-channel analysis of masked implementations. In *2018 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 57–64.
- [45] Fengchao Zhang, Andrew Hennessy, and Swarup Bhunia. 2015. Robust counterfeit PCB detection exploiting intrinsic trace impedance variations. In *2015 IEEE 33rd Vlsi Test Symposium (Vts)*. IEEE, 1–6.
- [46] YongBin Zhou and DengGuo Feng. 2005. Side-channel attacks: Ten years after its publication and the impacts on cryptographic module security testing. *Cryptology ePrint Archive*. Retrieved from <https://eprint.iacr.org/2005/388>
- [47] Huifeng Zhu, Haoqi Shan, Dean Sullivan, Xiaolong Guo, Yier Jin, and Xuan Zhang. 2023. PDNPulse: Sensing PCB anomaly with the intrinsic power delivery network. *IEEE Transactions on Information Forensics and Security* 18 (2023), 3590–3605.

Received 9 September 2024; revised 9 June 2025; accepted 22 August 2025

## 2 Related Work

This section provides a concise overview of the research advancements in HIFT (Section 2.1), the trace logic with different accuracies (Section 2.2), and the optimization of IFT (Section 2.3).

### 2.1 HIFT

Denning [9] first introduced the concept of IFT in 1976. IFT is a security analysis technique that creates an information flow model for the target system and analyzes how information propagates within it. Security labels are initially assigned to the information-carrying objects in the system to represent specific security attributes, and then security labels trace logic is introduced to enable tracking from the sensitive port to the release port. Security researchers can monitor the status of the output security labels and then determine whether the information complies to the specified security policy.

The confidentiality of the system is ensured by monitoring the status of security labels in the public domain to determine whether sensitive information has leaked into unclassified areas. The integrity of the system is verified by assessing whether the security label of an untrusted entity affects the label of a trusted entity, thereby indicating whether the trusted entity has been compromised.

Hardware information flow tracing can be employed to detect security vulnerabilities, including design flaws and bypass channels [14, 17, 27]. This approach provides an effective solution for verifying and testing hardware security. To achieve a balance between precision and scalability in hardware information flow, prior research has proposed security analysis at various abstraction levels of hardware design. These levels encompass gate-level [13, 19, 20, 26], cell-level [23], RTL level [4, 5], HDL level [3, 15, 16, 21, 28], and HLS level [8, 10, 22].

Information flow models implemented at various abstraction layers have different detection precision and verification performance. Gate-level information flow technology [26] is implemented by inserting trace logic into the gate-level netlist, providing high-precision tracking of information flow. However, the complexity of the model results in low verification performance, fails to meet scalability requirements, and presents certain issues with false positives. RTL level information flow technology [4] focuses on the integration of trace logic within **Hardware Description Languages (HDLs)**. This approach improves verification performance while maintaining precision. However, the complexity and abundance of high-level language structures may present challenges in fully processing these statements. Cell-level information flow technology [23] is deployed on the macrocell netlist between the above two abstraction layers and performs well in precision and verification performance.

### 2.2 Accuracy and Complexity of Trace Logic

HIFT technology integrates trace logic into the operational unit that processes hardware signals. Consider a two-input AND logic cell in the tracking model, where  $a_t$ ,  $b_t$ , and  $o_t$  represent the security labels of input and output, respectively. When the data input to port  $a$  contains confidential information, set  $a_t$  to 1. The security label  $o_t$  can be expressed using the appropriate trace logic according to the desired precision.

Information flow propagation logic can be represented as  $o_t = a_t \mid b_t$ , which conservatively captures all potential confidential information flows from input ports  $a$  or  $b$  to output port  $o$ . However, this propagation logic may be imprecise. For instance, if port  $a$  carries public data 0, which is  $a_t$  is 0 and port  $b$  carries confidential information 1, means  $b_t$  is 1, the output  $o$  will be dominated by  $a$ , and the other confidential input  $b$  has no effect on the output  $o$ . However,

this propagation logic conservatively expressed  $o_t = 1$ , classifying the output port as confidential. This conservative approach results in a significant number of false alarms, thereby increasing the workload for security personnel.

CellIFT [23] and GLIFT [26] provide a more precise method for computing the security label of output. The flow of information from input to output occurs only when the input value influences the output value. For an AND logic cell, its output port  $o$  security label can be expressed as  $o_t = b \& a_t \mid a \& b_t \mid a_t \& b_t$ . According to the above input values and security label, it is deduced that the output port security label  $o_t$  is 0, indicating that the exact propagation logic does not produce false alarms.

The higher the required tracking precision, the more complex the propagation logic becomes. To address this, Hu et al. [12] propose relaxing precision by progressively disregarding certain inputs, thereby moving toward a lower level of precision. This optimization results in a loss of precision in the propagation of hardware information flow. However, under specific constraints, functionally equivalent substitutions between imprecise and precise propagation strategies can be achieved.

### 2.3 IFT Optimization

Although hardware information flow tracing techniques can effectively detect logical vulnerabilities and verify the security properties of hardware designs, their inherent instrumentation overhead hinders the widespread application of this technology in the industry. In recent years, numerous researchers have proposed various optimization methods to reduce deployment overhead. These optimization techniques are generally classified into two categories: optimization of tracking logic and instrumentation areas.

*Optimization of the Instrumentation Area:* If static analysis can identify hardware regions that will never be involved in information flow propagation, instrumentation in those areas can be omitted. Previous work on optimizing IFT, conducted by Nicholas et al. [18], reduced deployment overhead by only tracking data from safety-critical modules. However, this granularity is too coarse, and the optimization remains insufficient. Chen et al. [6] reduced the deployment overhead of IFT in SelectiveTaint by dividing the must-not-taint directive. Zhang et al. [29] proposed the HardTaint optimization technique, which employs selective instrumentation and static analysis to achieve efficient hardware information flow tracing at the instruction architecture abstraction layer. The aforementioned methods utilize static analysis at various fine-grained levels to optimize information flow tracing within the instrumentation region. For large hardware designs, optimizing the instrumentation area can significantly reduce the deployment overhead associated with IFT.

*Optimization of Trace Logic:* Set lightweight information flow trace logic to reduce the complexity of the tracking model. Hu et al. [12, 24, 25] argue that the GLIFT logic for tracking information flow propagation contains redundancy under certain conditions. In the case of setting don't care input, replacing the complex GLIFT logic with simple propagation logic will not affect the precision. However, if the inputs influence the propagation of information flow as a don't care condition, the inaccurate propagation resulting from the substitution will introduce new false positives. Chen et al. [7] successfully simplified the tracking logic of multi-input cells from exponential complexity to linear complexity by introducing a “-” logic, which converts sum-of-products logic into product-of-sums logic. However, this method is not applicable to standard logic cells. Table 1 summarizes and compares the current work on IFT optimization.

## 3 Selective IFT

This section presents the deterministic constraint-driven IFT logic (Section 3.1) and the generation of tag-based selection information flow models (Section 3.2).

Table 1. Current Work on Information Flow Tracking Optimization

| Method   | Abstraction Level | Category  | Precision                                                      | Overhead    |
|----------|-------------------|-----------|----------------------------------------------------------------|-------------|
| [6]      | Binary level      | IA        | Accurate                                                       | Complex     |
| [29]     | Binary level      | IA        | Accurate                                                       | Complex     |
| [18]     | Gate level        | IA        | Ignore information flows from other modules                    | Lightweight |
| [24, 25] | Gate level        | TL        | Introduce false positives                                      | Lightweight |
| [12]     | Gate level        | TL        | Imprecision (overtainting)                                     | Complex     |
| [7]      | Gate level        | TL        | Imprecision (overtainting);<br>Applicable in certain scenarios | Lightweight |
| SHIFT    | RTL level         | TL and IA | Accurate                                                       | Lightweight |

TL, optimization of tracking logic; IA, optimization of instrumentation area.

### 3.1 IFT Logic

**3.1.1 Precision of Trace Logic.** By deploying information flow trace logic within the macrocell of hardware design, an information flow model is constructed to verify the system's confidentiality and integrity. This model assesses whether input signals influence output signals, and the precision of the trace logic affects the detection precision of the model. The imprecise information flow model employs a conservative least upper bound operator to determine the information flow, which can be represented as Equation (1):

$$L(O) = L(I_1) | L(I_2) | \dots | L(I_n). \quad (1)$$

The security labels of the outputs  $L(O)$  represent the OR operation of all input security labels, thereby implementing a lightweight yet imprecise information flow trace logic. Once the hardware design is synthesized into a netlist, the corresponding information flow trace logic is integrated. In the trace logic described in Equation (1), each netlist element incurs the overhead of one OR gate, resulting in a total overhead of  $n$  cells for a hardware design with  $n$  cells after synthesis.

The information flow trace logic presented in Equation (1) facilitates the rapid verification of potential security vulnerabilities with minimal instrumentation overhead. However, conservative verification results can lead to false positives, and troubleshooting these issues adds extra work for security analysts.

Precise information flow trace logic not only considers the security label associated with the input values but also incorporates the signal values  $I_n$  of the input into the propagation logic. In this trace logic, the security label  $L(O)$  is influenced by both the input value and the input security label. The precise information flow trace logic can be expressed as Equation (2):

$$L(O) = f\{I_1, I_2, \dots, I_n, L(I_1), L(I_2), \dots, L(I_n)\}. \quad (2)$$

Trace logic can accurately derive the information flow propagation based on different input values and security labels, but it introduces additional instrumentation overhead and verification time. The hardware design, after synthesizing into a netlist of  $n$  cells, has a total overhead of  $n \times x$  for the information flow model for the tracking logic of Equations (2) and (3), with an average cell-instrument overhead of  $x$ .

Unlike the OR operation, operation in Equations (1) and (2) introduces an information flow propagation function  $f$  based on the propagation characteristics of logic cells, enabling accurate but more complex IFT logic. However, this method only inserts different trace logic according to the cell type, providing generalized precision in IFT, which still results in significant redundancy overhead during actual deployment.

in SHIFT. The Systhesis Time metric reflects the generation time of the information flow model, including the static analysis time of SHIFT and the time of inserting staking tracking logic. The Verification Time metric reflects the verification time of the security attribute of the information flow model.

As shown in Figure 9, on the AES test set from Trust-Hub, compared with CellIFT, SHIFT reduces the deployment time of trace logic by 12.1%, the generated information flow model reduces the number of cells by 19.9% and the synthesis area by 35.7%, and reduces the security analysis time of the information flow model by 10.5%. The experimental results show that SHIFT can effectively alleviate the problem of excessive overhead in deploying IFT techniques on large and complex hardware designs.

## 6 Summarize

This article presents a constraint-driven optimization technique for SHIFT. In terms of the capability to trace information flow, the trace logic based on macrocells offers superior precision and performance compared to other abstraction layers. In addition, to reduce the instrumentation overhead, SHIFT adapts optimization techniques from the field of software information flow tracing to mitigate redundant logic in hardware information flow tracing models. Specifically, coarse-grained static analysis can efficiently extract constraints and selectively deploy trace logic when generating trace models. In comparison to previous optimization techniques, SHIFT demonstrates advancements in both optimization efficiency and scalability. Experimental results across various hardware designs indicate that SHIFT can reduce the overhead associated with deploying tracking logic without increasing false positives, thereby achieving an accurate and lightweight HIFT technology.

However, this work necessitates that security personnel provide the initial taint value, and the coarse-grained static analysis may still introduce some redundancy in the IFT model. To address the former issue, we propose the automated deployment of taint initial values by leveraging large language models in the future. For the latter issue, we can consider incorporating additional logic into the instrumentation process to capture dynamic runtime constraint information, which will result in a more efficient IFT model.

## References

- [1] OpenCores. 2025. Open Source IP-Cores. Retrieved from <https://opencores.org/projects?expanded=Crypto%20core>
- [2] Trust-Hub. 2025. Trust-Hub.org. Retrieved from <https://trust-hub.org/#/benchmarks/chip-level-trojan>
- [3] Armaiti Ardeshiricham, Wei Hu, and Ryan Kastner. 2017. Clepsydra: Modeling timing flows in hardware designs. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 147–154.
- [4] Armaiti Ardeshiricham, Wei Hu, Joshua Marxen, and Ryan Kastner. 2017. Register transfer level information flow tracking for provably secure hardware design. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1691–1696.
- [5] Armaiti Ardeshiricham, Yoshiki Takashima, Sicun Gao, and Ryan Kastner. 2019. VeriSketch: Synthesizing secure hardware designs with timing-sensitive information flow properties. In *2019 ACM SIGSAC Conference on Computer and Communications Security*, 1623–1638.
- [6] Sanchuan Chen, Zhiqiang Lin, and Yinqian Zhang. 2021. {SelectiveTaint}: Efficient data flow tracking with static binary rewriting. In *30th USENIX Security Symposium (USENIX Security '21)*, 1665–1682.
- [7] Yongliang Chen, Xiaole Cui, Xiaoxin Cui, and Xing Zhang. 2024. The area-efficient gate level information flow tracking schemes of digital circuit with multi-level security lattice. *Microelectronics Journal* 144 (2024), 106088.
- [8] Shuwen Deng, Doğuhan Gümüsoğlu, Wenjie Xiong, Y. Serhan Gener, Onur Demir, and Jakub Szefer. 2017. Secchisel: Language and tool for practical and scalable security verification of security-aware hardware architectures. Cryptology ePrint Archive, Report 2017/193.
- [9] Dorothy E. Denning. 1976. A lattice model of secure information flow. *Communications of the ACM* 19, 5 (May 1976), 236–243. DOI: <https://doi.org/10.1145/360051.360056>
- [10] Mehran Goli and Rolf Drechsler. 2021. ATLaS: Automatic detection of timing-based information leakage flows for SystemC HLS designs. In *26th Asia and South Pacific Design Automation Conference*, 67–72.

- [11] Wei Hu, Armaity Ardeshircham, and Ryan Kastner. 2021. Hardware information flow tracking. *ACM Computing Surveys* 54, 4, Article 83 (May 2021), 39 pages. DOI: <https://doi.org/10.1145/3447867>
- [12] Wei Hu, Andrew Becker, Armita Ardeshircham, Yu Tai, Paolo Ienne, Dejun Mu, and Ryan Kastner. 2016. Imprecise security: Quality and complexity tradeoffs for hardware information flow tracking. In *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [13] Wei Hu, Baolei Mao, Jason Oberg, and Ryan Kastner. 2016. Detecting hardware trojans with gate-level information-flow tracking. *Computer* 49, 8 (2016), 44–52.
- [14] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. 2020. Spectre attacks: Exploiting speculative execution. *Communications of the ACM* 63, 7 (Jun. 2020), 93–101. DOI: <https://doi.org/10.1145/3399742>
- [15] Xun Li, Vineeth Kashyap, Jason K. Oberg, Mohit Tiwari, Vasanth Ram Rajarathinam, Ryan Kastner, Timothy Sherwood, Ben Hardekopf, and Frederic T. Chong. 2014. Sapper: A language for hardware-level security policy enforcement. In *19th International Conference on Architectural Support for Programming Languages and Operating Systems*, 97–112.
- [16] Xun Li, Mohit Tiwari, Jason K. Oberg, Vineeth Kashyap, Frederic T. Chong, Timothy Sherwood, and Ben Hardekopf. 2011. Caisson: A hardware description language for secure information flow. *ACM SIGPLAN Notices* 46, 6 (2011), 109–120.
- [17] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, et al. 2020. Meltdown: Reading kernel memory from user space. *Communications of the ACM* 63, 6 (May 2020), 46–56. DOI: <https://doi.org/10.1145/3357033>
- [18] Geraldine Shirley Nicholas, Dhruvakumar Vikas Aklekar, Bhavin Thakar, and Fareena Saqib. 2023. Secure instruction and data-level information flow tracking model for risc-v. *Cryptography* 7, 4 (2023), 58.
- [19] Jason Oberg, Wei Hu, Ali Irturk, Mohit Tiwari, Timothy Sherwood, and Ryan Kastner. 2010. Theoretical analysis of gate level information flow tracking. In *47th Design Automation Conference*, 244–247.
- [20] Jason Oberg, Sarah Meiklejohn, Timothy Sherwood, and Ryan Kastner. 2014. Leveraging gate-level properties to identify hardware timing channels. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33, 9 (2014), 1288–1301.
- [21] Maoyuan Qin, Xinmu Wang, Baolei Mao, Dejun Mu, and Wei Hu. 2020. A formal model for proving hardware timing properties and identifying timing channels. *Integration* 72 (2020), 123–133.
- [22] Fabian Schuiki, Andreas Kurth, Tobias Grosser, and Luca Benini. 2020. LLHD: A multi-level intermediate representation for hardware description languages. In *41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 258–271.
- [23] Flavien Solt, Ben Gras, and Kaveh Razavi. 2022. {CellIFT}: Leveraging cells for scalable and precise dynamic information flow tracking in. {RTL}. In *31st USENIX Security Symposium (USENIX Security '22)*, 2549–2566.
- [24] Yu Tai, Wei Hu, Dejun Mu, Baolei Mao, Lantian Guo, and Maoyuan Qin. 2018. A simplifying logic approach for gate level information flow tracking. In *12th International Conference on Communications and Networking*. Springer, 302–311.
- [25] Yu Tai, Wei Hu, Hui-Xiang Zhang, De-Jun Mu, and Xing-Li Huang. 2016. Generating optimized gate level information flow tracking logic for enforcing multilevel security. *Automatic Control and Computer Sciences* 50 (2016), 361–368.
- [26] Mohit Tiwari, Hassan, M. G. Wassel, Bitu Mazloom, Shashidhar Mysore, Frederic T. Chong, and Timothy Sherwood. 2009. Complete information flow tracking from the gates up. In *14th International Conference on Architectural Support for Programming Languages and Operating Systems*, 109–120.
- [27] Ofir Weisse, Jo Van Bulck, Marina Minkin, Daniel Genkin, Baris Kasicki, Frank Piessens, Mark Silberstein, Raoul Strackx, Thomas, F. Wenisch, and Yuval Yarom. 2018. *Foreshadow-NG: Breaking the Virtual Memory Abstraction with Transient out-of-Order Execution*. Technical report.
- [28] Danfeng Zhang, Yao Wang, G. Edward Suh, and Andrew C. Myers. 2015. A hardware design language for timing-sensitive information-flow security. *ACM SIGPLAN Notices* 50, 4 (2015), 503–516.
- [29] Yiyu Zhang, Tianyi Liu, Yueyang Wang, Yun Qi, Kai Ji, Jian Tang, Xiaoliang Wang, Xuandong Li, and Zhiqiang Zuo. 2024. HardTaint: Production-run dynamic taint analysis via selective hardware tracing. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 1615–1640.

Received 18 January 2025; revised 5 June 2025; accepted 22 August 2025

CCS Concepts: • **Hardware** → **Dynamic memory; Memory and dense storage; Computer systems organization** → *Distributed architectures*;

Additional Key Words and Phrases: Analog memory, Artificial Neural Network (ANN), In-Memory Computation (IMC), Matrix-Vector Multiplication (MVM), On-chip training

#### ACM Reference format:

Shreyas Deshmukh, Raghav Singhal, Shruti Landge, Vivek Saraswat, Anmol Biswas, Abhishek Kadam, Ajay K. Singh, Sreenivas Subramoney, Laxmeesha Somappa, Maryam Shojaei Baghini, and Udayan Ganguly. 2025. Analog and Temporary On-chip Memory for ANN Training and Inference. *ACM J. Emerg. Technol. Comput. Syst.* 21, 4, Article 14 (October 2025), 18 pages.  
<https://doi.org/10.1145/3765899>

## 1 Introduction

**Artificial neural networks (ANNs)** have become a prominent candidate for **machine learning (ML)** applications like image classification [47], face recognition [2], and speech processing [30]. For these applications, hardware systems at the edge of the cloud or in autonomous (cloud-free) environments prefer the on-chip/*in-situ* ANN training [24, 26]. On-chip training at the edge has certain advantages: first, latency, energy, and data privacy improvements due to the avoidance of cloud communication or when the cloud is unavailable [24]. Second, on-chip training is variability-aware and suitable for dynamic environments because it can adapt to the operational setup (e.g., voltage supply variation), environment (e.g., temperature, noise, etc.), and manufacturing non-idealities (e.g., transistor mismatch, variability) [11, 24].

**In-memory computation (IMC)** is a popular choice to accelerate ANN training and inference as it has reduced area, power, and latency requirements compared to conventional Von-Neuman architecture [10, 21, 29, 37, 44]. The analog IMC with a bi-directional memory array has been demonstrated to accelerate the ANN training using a back-propagation algorithm [4, 17, 41]. Some analog IMC-based **multiply-accumulate (MAC)** operation unit accepts digital inputs and generate analog outputs. It requires **analog-to-digital converters (ADCs)** with a high area and power penalty to convert the MAC output to digital form for the following ANN layers [15, 20, 28]. Therefore, the challenge is to design ANN hardware with reduced ADC requirements.

In addition, extensive intermediate data is generated during the ANN training's **forward pass (FP)** and **backward pass (BP)** operations. This intermediate data, i.e., neuronal activation, error gradient, and weight gradient, is temporarily required during training and conventionally stored in off-chip digital memory [6, 17, 34]. However, off-chip communication cost (i.e., area, power, and latency) is remarkably high [12, 35]. Moreover, ADCs are required before off-chip communication of intermediate data because IMC outputs, which partially represent intermediate data, are analog. Therefore, analog on-chip intermediate data storage is another challenge.

In this article, we offer a solution to these challenges of reduced ADC requirements and on-chip intermediate data storage for ANN on-chip training hardware. We propose an **analog temporary on-chip memory (ATOM)** to store intermediate data in the analog domain and surmount conventional on-chip training challenges with off-chip intermediate data storage. The proposed ATOM uses a field-effect transistor gate capacitor to store the intermediate data in an analog domain. We have presented the ATOM unit cell characteristics from 20 fabricated chips. In addition, we have analyzed the impact of analog memory storage on ANN system performance.

Table 1 qualitatively presents the status of the available and proposed intermediate data storage cells for ANN training and inference. Off-chip DRAM [17] and on-chip SRAM [34] have been commonly used to store intermediate data; however, ADCs are required to convert the analog

Table 1. Status of the Available Intermediate Data Storage Cells for ANN Training and Inference

|                            | [17]     | [34]     | [31]      | [4]      | [15]      | Proposed |
|----------------------------|----------|----------|-----------|----------|-----------|----------|
| Application                | Training | Training | Inference | Training | Inference | Training |
| On-chip Memory             | No       | Yes      | Yes       | Yes      | Yes       | Yes      |
| Analog Memory              | No       | No       | No        | Yes      | Yes       | Yes      |
| Experimental Demonstration | No       | Yes      | No        | No       | Yes       | Yes      |
| Retention Analysis         | NA       | NA       | Yes       | No       | No        | Yes      |
| Variability Analysis       | NA       | NA       | NA        | No       | No        | Yes      |

MAC outputs before storing them in digital memories. Recently, an innovative mixed CMOS cell memory design has been introduced to optimize performance, area, and energy efficiency for intermediate data storage by combining SRAM and eDRAM cells [31]. However, ADCs remain necessary during AI operations to convert intermediate data into a digital format for storage in the on-chip digital memory. Moreover, on-chip analog memories have been explored for training [4] and inference [15], but challenges such as memory decay, variability, and their impact on system performance need further consideration. Despite the benefits of analog memory and its processing, such as power and area efficiency and quantization-free processing [15, 16], non-idealities like variability and capacitor discharge-induced decay can hinder performance, unlike in digital domain implementations. Therefore, in this article, to address the challenge of area- and energy-hungry ADC usage during ANN training and inference, we propose ATOM for intermediate data storage and evaluate the robustness of on-chip training network performance against the effects of decay and variability in ATOM.

Since ATOM stores intermediate data in the analog domain, it is particularly advantageous when paired with the analog IMC unit, which accepts the activation inputs in the analog domain. The IMC architectures are categorized based on whether the accumulation operation occurs in the analog or digital domain as analog and digital IMC [18, 19, 29, 44]. The literature proposes and demonstrates the charge-based [8, 14, 25, 46] and current-based [13, 17, 41] analog IMC units. Specific analog IMC units (both current- and charge-based) accept analog input [13, 15, 25], while others require inputs in the digital form [14, 17, 46]. However, the output from all analog IMC units will always be in analog form. Consequently, for analog IMC, when inputs are accepted in the analog domain, the need for ADCs can be avoided by storing the MAC output in addition to other intermediate data (i.e., error gradient and weight gradient) in the analog domain using ATOM. However, for analog IMC architectures that accept digital input, ADCs are essential because the output from the previous layer (or previous IMC unit) will be in analog form. Therefore, the proposed ATOM can eliminate the need for ADCs during intermediate data storage for ANN architecture with an analog IMC unit (both current- and charge-based), which accepts the input in analog form.

In [40], we have presented another application of analog memory, i.e., a memory element, to store the weights in the IMC unit. While [40] highlights the importance of analog memory as a synapse (weight) within the context of IMC operations in ANN, this article demonstrates the role of analog memory as a neuron component, specifically for storing intermediate data generated during ANN inference and training. In [40], we showed that analog volatile memory for synapses enhances inference performance accuracy, as the natural decay of this memory acts as a form of regularization, helping to mitigate the challenge of overfitting during ANN training. On the other hand, analog memory demonstrated in this article for intermediate data storage addresses the challenges of off-chip data communication and the costly analog-to-digital conversion inherent in traditional ANN training architectures [17].

In summary, the ATOM characterization presented above provides the retention time constant, in milliseconds, for a given  $V_{in}$  range (0.4 V to 0.9 V). Moreover, the switch using M2 is used for the read operation, which results in no static power with the energy needed for a read operation as  $\sim 1.6$  pJ.

Now, the adequacy of ATOM parameters for intermediate data storage during ANN training must be evaluated. Therefore, the impact of ATOM parameters on the performance of the ANN training system is discussed in the following section.

## 5 System Performance Evaluation

We simulated the system in PyTorch to analyze the impact of ATOM parameters on system performance during ANN training. The variations in ATOM retention time are applied to the memory, which is responsible for storing intermediate data. As we aim to analyze the impact of ATOM on system accuracy performance, all remaining circuits involved in ANN training are assumed to be immune to variations and non-idealities. The subsequent section describes the simulation model utilized to evaluate and analyze system performance with the ATOM retention effect.

### 5.1 Performance Evaluation

To evaluate system performance, we have used various network architectures, including AlexNet [23] and VGG [39], and different datasets like MNIST, CIFAR-10, and CIFAR-100 [22]. The models were trained using the SGD optimizer [27] with a momentum of 0.9, an initial learning rate of 0.05, and a batch size of 256. We train the model using a softmax cross-entropy loss and run it for 200 epochs. All our results are averaged over 5 different runs.

The training and inference are performed using a MAC with 8-bit quantized weights. For the evaluation model, we consider the SRAM memory array for IMC MAC with 7T SRAM bit-cells to store the weights and to perform bidirectional MAC [2, 17]. The analog shift-add scheme, discussed in [16], is considered in the periphery to obtain the weighted-bit precision addition. Our previous work [40] modeled ATOM's effects for storing the weights and biases. In this work, we store the intermediate memory components, i.e., activations, errors, and updates, in ATOM. The appropriate time to perform the compute or memory operations, i.e., operation time constant ( $\tau_{op}$ ), is taken to be 10 ns.

As shown in Figure 7(a), we investigated the impact of varying  $\tau_{ret}$  across a broad range, from  $10^3$  ms to  $10^{-6}$  ms, for various network architectures, including AlexNet [23] and VGG [39] with the CIFAR-10 dataset [22]. We observe minimal performance degradation over a significant span of  $\tau_{ret}$  values, specifically when  $\tau_{ret} \geq 10^{-4}$  ms. As can be seen in Figure 7(b), for  $\tau_{ret} \geq 10^{-4}$  ms, the accuracy remains comparable to the non-ATOM scenario, i.e., for ideal analog memory with infinite  $\tau_{ret}$  or no decay. Moreover, a mild regularization effect is observed in Figure 7(b), as the ATOM decay effectively serves as a weak regularizer within a suitable range [40]. For  $\tau_{ret} < 10^{-4}$  ms, the training starts to become highly unstable due to extremely high regularization and fails to converge (i.e., validation accuracy drops to 10%), as can be observed in Figure 7(a).

As seen from Figure 7(a) and (c), with the depth of the network (from AlexNet to VGG-19), the required retention time to store intermediate data increases. As ATOM needs to store the intermediate data during ANN training (for the given input sample), the required retention time increases with the depth of the neural network (i.e., the number of layers). The deeper the network, the longer the ATOM retention time must be since the intermediate data from the first layer is needed until the FP (from the first to the last layer) and the BP (from the last to the first layer) are completed for the given input sample. The required retention time for the VGG-19 network with 19 layers is higher than the AlexNet network (with eight layers), as can be seen in Figure 7(c).

Table 3. System Level Performance Accuracy and Operational Complexity of Neural Network Architecture with ATOM and with Different Memory Technologies (Presented in Literature) for Intermediate Data Storage

|                                                 |                                       | TComp'20<br>[17] | ISCA'16<br>[38] | JSSC'22<br>[34] | TVLSP'24<br>[31]                              | Proposed |          |           |
|-------------------------------------------------|---------------------------------------|------------------|-----------------|-----------------|-----------------------------------------------|----------|----------|-----------|
| Memory Technology for Intermediate Data Storage |                                       | DRAM             | eDRAM           | SRAM            | SRAM + eDRAM                                  | ATOM     |          |           |
| Technology Node                                 |                                       | 7 nm             | 32 nm           | 40 nm           | 45 nm                                         | 45 nm    |          |           |
| Neural Network Architecture                     |                                       | ResNet-18        | CNN/<br>DNN     | ResNet-18       | DNN, MLP,<br>and GAN                          | VGG-13   |          |           |
| Dataset                                         |                                       | ImageNet         | — <sup>a</sup>  | ImageNet        | MNIST,<br>CIFAR-10,<br>CIFAR-100,<br>ImageNet | MNIST    | CIFAR-10 | CIFAR-100 |
| Classification<br>Accuracy (%)                  | Baseline <sup>b</sup>                 | — <sup>a</sup>   | — <sup>a</sup>  | ~75             | Relative<br>accuracy<br>close to 1            | 99.34    | 93.81    | 73.81     |
|                                                 | With<br>Measured<br>Data <sup>c</sup> |                  |                 |                 |                                               | 99.29    | 93.14    | 72.59     |
| FLOPs                                           |                                       | — <sup>a</sup>   | — <sup>a</sup>  | — <sup>a</sup>  | — <sup>a</sup>                                | 335 M    | 494 M    | 495 M     |
| MACs<br>Operations                              |                                       | — <sup>a</sup>   | — <sup>a</sup>  | — <sup>a</sup>  | — <sup>a</sup>                                | 167 M    | 247 M    | 248 M     |

<sup>a</sup>Not explicitly mentioned.<sup>b</sup>System level performance accuracy with ideal analog intermediate data storage memory (without decay over time).<sup>c</sup>System level performance accuracy with experimentally measured retention of ATOM that is 0.66 ms.

input image dimensions for CIFAR-10 and CIFAR-100 result in the overall FLOPs and MACs being nearly the same across both datasets. However, the slight rise in FLOPs and MACs, especially in the fully connected layers of VGG-13, for CIFAR-100 is due to an increased number of output classes (100 classes). As depicted in Table 3, the performance accuracy of the VGG-13 network decreases from MNIST to CIFAR-100, highlighting the impact of the growing complexity of the datasets on system accuracy performance.

Next, we analyze the impact of variability in the ATOM's retention time constant on the system performance with CIFAR-10 datasets and the VGG-13 neural network architecture. We use a fixed value of  $\tau_{ret} = 1$  ms, which lies comfortably within the ideal range of  $\tau_{ret}$  where we observe the slight regularization benefit, as can be seen in Figure 7(b). The leakage current variation of the write transistor M6 of ATOM unit cells dominates cell-to-cell variability in the retention time constant. The threshold voltage of M6, i.e.,  $V_{Th,M6}$ , is Gaussian distributed, and  $\tau_{ret}$  exponentially depends on  $V_{Th,M6}$ . Thus,  $\tau_{ret}$  is sampled from a log-normal distribution:

$$\tau_{ret,0} \cdot e^{\left(\frac{\Delta V_{Th,M6}}{\eta V_t}\right)},$$

where  $\tau_{ret,0}$  is the nominal value of  $\tau_{ret}$ ,  $\Delta V_{Th,M6}$  is a Gaussian random variable with zero mean and standard deviation  $\sigma_{V_{Th,M6}}$ .  $V_t$  is the thermal voltage (26 mV at room temperature), and  $\eta$  is an ideality factor ( $\sim 1.5$ ).

We conducted simulations for various values of  $\sigma_{V_{Th}}$  for M6 of ATOM, denoted as  $\sigma_{V_{Th,M6}}$ , and tracked the test accuracies, as shown in Table 4. Our observation reveals either no or minimal performance decline for  $\sigma_{V_{Th,M6}}$  values up to 80 mV. This suggests performance robustness against the variability in ATOM's retention time. Consequently, we conclude that on-chip training using the proposed ATOM remains robust in the face of on-chip variations and charge loss over time within ATOM.

Therefore, based on the results illustrated in Figure 7 and Table 3, we can conclude that ATOM, when configured with an appropriate retention time constant, can be applied effectively across

Table 4. Accuracy Evaluation of VGG-13 Neural Network Architecture with ATOM and CIFAR-10 Datasets for Various  $\sigma_{V_{Th,M6}}$  Values for  $\tau_{ret} = 1$  ms

| $\sigma_{V_{Th,M6}}$ (mV) | Test Accuracy (%) |
|---------------------------|-------------------|
| 10                        | $93.18 \pm 0.16$  |
| 20                        | $93.10 \pm 0.05$  |
| 40                        | $93.28 \pm 0.05$  |
| 80                        | $93.23 \pm 0.13$  |

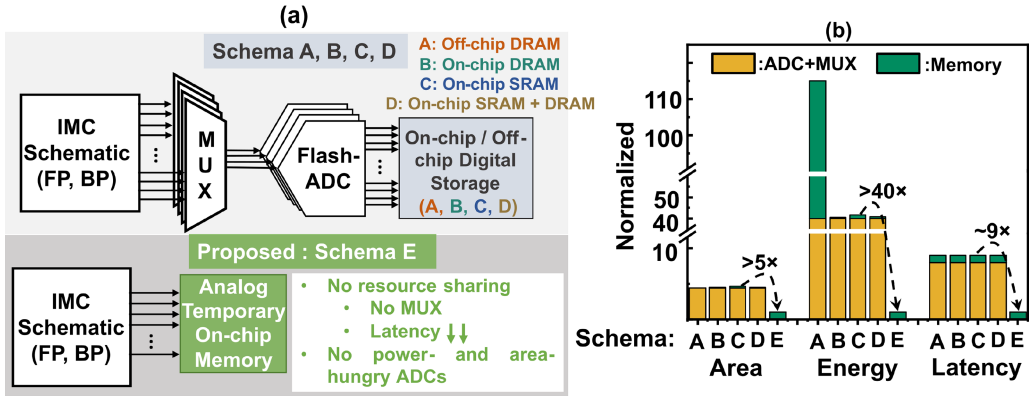


Fig. 8. Comparison between conventional and proposed schema for FP and BP operation of ANN training: (a) Conventional and proposed schema for intermediate data storage during FP and BP. (b) Comparison between various schemas for intermediate data storage during FP and BP.

various datasets and neural network architectures. Furthermore, as depicted in Figure 7, the required time constant for the presented neural network architecture can be minimized up to  $\tau_{ret} \approx 10^{-4}$  ms without affecting system performance; it enables the reduction in the ATOM cell area (at least by  $2\times$ , as presented in Figure 5). This area reduction allows for more memory cells to be integrated within a given memory area. Moreover, as highlighted in Table 4, on-chip training using ATOM for intermediate data storage exhibits robustness against on-chip variations and charge loss over time within the ATOM.

## 5.2 Hardware Benefits

This subsection compares the area, energy consumption, and number of clock cycles (i.e., latency) required during FP and BP for intermediate data storage. The FP and BP operation in the schema with ATOM against various conventional schemas that utilize digital memory for intermediate data storage is shown in Figure 8(a). For a baseline, we considered different intermediate data storage memory technologies presented in the literature, including off-chip DRAM [17], on-chip DRAM (eDRAM) [38, 43, 48], SRAM [34], and SRAM+eDRAM [31], and labeled them as Schemas A, B, C, and D, respectively.

As shown in Figure 8(a), Schemas A, B, C, and D store the large volumes of intermediate data generated during ANN inference and training in digital memory. They rely on energy- and area-intensive ADCs to store intermediate data in digital memory because IMC output (which represents

Table 5. Estimated Resources for Intermediate Data Storage at Macro-level ( $128 \times 128$  Memory Array) by an ANN Hidden Layer with Various Schema during FP and BP Operation of Inference and Training (Estimated at 45 nm Technology Node)

| Schema                                    |           | A: DRAM<br>[17] | B: eDRAM<br>[38, 43, 48] | C: SRAM<br>[34] | D: SRAM +<br>eDRAM<br>[31] | E: ATOM<br>(Proposed) |
|-------------------------------------------|-----------|-----------------|--------------------------|-----------------|----------------------------|-----------------------|
| Area (mm <sup>2</sup> )                   | ADC + MUX | 0.056           | 0.056                    | 0.056           | 0.056                      | NA <sup>a</sup>       |
|                                           | Storage   | -- <sup>b</sup> | 0.001                    | 0.003           | 0.002                      | 0.012                 |
|                                           | Total     | -- <sup>b</sup> | 0.057                    | 0.059           | 0.058                      | 0.012                 |
| Energy (nJ)                               | ADC + MUX | 16.5            | 16.5                     | 16.5            | 16.5                       | NA <sup>a</sup>       |
|                                           | Storage   | 30.7            | 0.18                     | 0.64            | 0.24                       | 0.41                  |
|                                           | Total     | 46.7            | 16.6                     | 17.1            | 16.7                       | 0.41                  |
| Latency<br>(No. of clock<br>cycles)       | ADC + MUX | 8               | 8                        | 8               | 8                          | NA <sup>a</sup>       |
|                                           | Storage   | 1               | 1                        | 1               | 1                          | 1                     |
|                                           | Total     | 9               | 9                        | 9               | 9                          | 1                     |
| Throughput (GOPS) <sup>c,d</sup>          |           | ~102            | 128                      | 128             | 128                        | ~171                  |
| Energy Efficiency (TOPS/W) <sup>c,d</sup> |           | 5.36            | 14.30                    | 13.95           | 14.26                      | 124.57                |

<sup>a</sup>Not Applicable: As ADCs and MUX are not required in the proposed schema.

<sup>b</sup>Off-chip DRAM area is not considered here (as a separate IC is required for DRAM).

<sup>c</sup>Resources required to perform MAC operation are taken into account in addition to intermediate storage.

<sup>d</sup>1 Operation = 1-bit Weight  $\times$  analog Input.

GOPS represents Giga-operations Per Second, whereas TOPS/W represents Tera-operations Per Second per Watt.

a significant portion of intermediate data) is in the analog domain. To minimize the number of necessary ADCs, a multiplexer (MUX) is used to share one ADC between eight IMC analog outputs [17]. Although this reduces the area requirement, it increases the latency or the required number of clock cycles. Moreover, Schema A requires energy-hungry off-chip communication to store the intermediate data to off-chip digital memory. To reduce the area and power costs of ADCs, some studies suggest using analog memory for intermediate data storage [4, 15]. However, in [15], analog memory is demonstrated only for inference operations. Chang et al. [4] offer analog memory for inference and training, but its hardware has not been experimentally verified, and no detailed information about the analog memory retention requirement is available. The proposed ATOM for intermediate data storage (Schema E) does not require analog-to-digital conversion and minimizes off-chip communication for intermediate data storage, which are challenges associated with existing schemas. The benefits of this approach, in terms of area, energy, latency, throughput, and energy efficiency, are summarized in Table 5 for FP and BP operations of ANN training.

For comparison, presented in Table 5 and Figure 8(b), we have considered an ANN hidden layer with an IMC schematic using eight tiles of  $128 \times 128$  memory crossbar array [17]. The parameters in Table 5 are estimated at the macro level for a  $128 \times 128$  memory crossbar array macro, which serves as a foundational building block for ANNs. This macro efficiently performs MVM operations for inference and training of ANN using IMCs independent of the underlying network architecture or dataset [17, 33]. Therefore, the parameters in Table 5 are valid for all network architectures (e.g., AlexNet, VGG-13, VGG-16, and VGG-19) and datasets (e.g., MNIST, CIFAR-10, and CIFAR-100) discussed in this article and are not limited to just these examples.

Conventional memory technology, such as SRAM [17] or emerging memory technology, e.g., RRAM [9, 15, 36], can be used in this  $128 \times 128$  memory crossbar array macro to perform the IMC operation. The resources required by the memory, ADCs, and MUX to store the intermediate data

generated by the ANN hidden layer for various schemas (shown in Figure 8(a)) are compared in Table 5. As the IMC schematic is common for all schemas and can use either SRAM or RRAM crossbar array, it is not considered in area, energy, and latency estimation presented in Table 5 and Figure 8(b) [31]. However, it is considered in the analysis of throughput and energy efficiency.

As shown in Figure 8(b), the proposed schema (Schema E) offers a significant improvement compared to on-chip SRAM (Schema C) for intermediate data storage, with a  $\sim 5\times$  reduction in area, a  $\sim 40\times$  reduction in energy, and a  $\sim 9\times$  decrease in the required number of clock operation for FP and BP. This is because the primary resource-consuming factor in a conventional schema is an ADC, with an estimated area of  $\sim 0.007 \text{ mm}^2$  and energy of  $\sim 0.065 \text{ nJ}$  for 45 nm technology [16], which is entirely avoided in the proposed Schema E for FP and BP. Moreover, off-chip communication is a significant source of energy consumption in Schema A. However, in the proposed Schema E, this issue is significantly mitigated by utilizing ATOM to store a substantial volume of intermediate data, minimizing the need for off-chip communication. The off-chip DRAM communication is the costliest operation, which takes nearly 120 pJ/B [12, 26], compared to 0.7 pJ/B for eDRAM [26], 2.5 pJ/B for on-chip SRAM [12], 0.925 pJ/B for SRAM+eDRAM [31], and 1.6 pJ for the proposed ATOM. Further, the proposed Schema E achieves a latency improvement of  $\sim 9\times$  as multiplexers are not required at the periphery compared to other schemas. The throughput and energy efficiency (in TOPS/W) are estimated for all schemas, considering a 45 nm technology node and an operating frequency of 500 MHz. Schema A has lower throughput and energy efficiency than other schemas due to off-chip communication in addition to ADC usage. However, higher throughput and energy efficiency can be achieved with Schema E, as no ADCs and off-chip communication are required during FP and BP for intermediate data storage compared to other schemas.

Therefore, with its significant advantages in energy, latency, throughput, and overall performance, primarily resulting from the elimination of ADC usage during FP and BP, Schema E, which integrates the IMC unit with the proposed ATOM, presents a highly compelling solution for on-chip training. Schema E surpasses traditional digital memory solutions for intermediate data storage, utilizing analog data storage with ATOM, providing a more efficient and effective alternative for ANN applications.

## 6 Conclusion

We present a novel ATOM prototype for intermediate data storage during on-chip training and inference of ANNs in the analog environment. Our proposal's key concept is maintaining the intermediate data storage on-chip and in the analog domain, leading to ANN training with minimum ADCs. Furthermore, system hardware variability awareness in ANN, which improves performance accuracy, is achieved using on-chip training. Our proposed analog memory is designed to complement the operation in the analog domain. Our approach avoids needing area-intensive and power-hungry ADCs and memories compared to conventional intermediate data storage schemas. The proposed schema yields at least  $5\times$  improvement in the area,  $40\times$  in energy, and  $9\times$  in latency.

## References

- [1] Nihar Athreya, Wenhao Song, Blair Perot, Qiangfei Xia, Abbie Mathew, Jai Gupta, Dev Gupta, and J. Joshua Yang. 2018. Memristor-CMOS analog coprocessor for acceleration of high-performance computing applications. *Journal on Emerging Technologies in Computing Systems* 14, 3 (Nov. 2018), Article 38, 30 pages. DOI: <https://doi.org/10.1145/3269985>
- [2] Kyeongryeol Bong, Sungpill Choi, Changhyeon Kim, Sanghoon Kang, Youchang Kim, and Hoi-Jun Yoo. 2017. 14.6 A 0.62mW ultra-low-power convolutional-neural-network face-recognition processor and a CIS integrated with always-on Haar-like face detector. In *2017 IEEE International Solid-State Circuits Conference (ISSCC)*, 248–249. DOI: <https://doi.org/10.1109/ISSCC.2017.7870354>

- [3] Lylia Thiziri Chabane, Dang-Kiên Germain Pham, Paul Chollet, and Patricia Desgreys. 2021. Design method of analog sigmoid function and its approximate derivative. In *2021 XXXVI Conference on Design of Circuits and Integrated Systems (DCIS)*, 1–5. DOI : <https://doi.org/10.1109/DCIS53048.2021.9666181>
- [4] H.-Y. Chang, P. Narayanan, S. C. Lewis, N. C. P. Farinha, K. Hosokawa, C. Mackin, H. Tsai, S. Ambrogio, A. Chen, and G. W. Burr. 2019. AI hardware acceleration with analog memory: Microarchitectures for low energy at high speed. *IBM Journal of Research and Development* 63, 6 (2019), 8:1–8:14. DOI : <https://doi.org/10.1147/JRD.2019.2934050>
- [5] Ethan Chen and Vanessa Chen. 2020. In-sensor time-domain classifiers using pseudo sigmoid activation functions. *Integration* 73 (2020), 43–49. DOI : <https://doi.org/10.1016/j.vlsi.2020.03.002>
- [6] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. 2017. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits* 52, 1 (2017), 127–138. DOI : <https://doi.org/10.1109/JSSC.2016.2616357>
- [7] Zhengyu Chen, Xi Chen, and Jie Gu. 2021. 15.3 A 65nm 3T dynamic analog RAM-based computing-in-memory macro and CNN accelerator with retention enhancement, adaptive analog sparsity and 44TOPS/W system energy efficiency. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64, 240–242. DOI : <https://doi.org/10.1109/ISSCC42613.2021.9366045>
- [8] Zhiyu Chen, Zhanghao Yu, Qing Jin, Yan He, Jingyu Wang, Sheng Lin, Dai Li, Yanzhi Wang, and Kaiyuan Yang. 2021. CAP-RAM: A charge-domain in-memory computing 6T-SRAM for accurate and precision-programmable CNN Inference. *IEEE Journal of Solid-State Circuits* 56, 6 (June 2021), 1924–1935. DOI : <https://doi.org/10.1109/JSSC.2021.3056447>
- [9] Shreyas Deshmukh, Vivek Saraswat, Venkatesh Gopinath, Rajesh Nair, Laxmeesha Somappa, Maryam S. Baghini, and Udayan Ganguly. 2023. ANN Inference enabled by variability mitigation using 2T-1R bit cell-based design space analysis. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 1–5. DOI : <https://doi.org/10.1109/ISCAS46773.2023.10181912>
- [10] Anteneh Gebregiorgis, Hoang Anh Du Nguyen, Jintao Yu, Rajendra Bishnoi, Mottaqiallah Taouil, Francky Catthoor, and Said Hamdioui. 2022. A survey on memory-centric computer architectures. *Journal on Emerging Technologies in Computing Systems* 18, 4 (Oct. 2022), Article 79, 50 pages. DOI : <https://doi.org/10.1145/3544974>
- [11] Sujan Kumar Gonugondla, Mingu Kang, and Naresh Shanbhag. 2018. A 42pJ/decision 3.12TOPS/W robust in-memory machine learning classifier with on-chip training. In *2018 IEEE International Solid-State Circuits Conference (ISSCC)*, 490–492. DOI : <https://doi.org/10.1109/ISSCC.2018.8310398>
- [12] Mark Horowitz. 2014. 1.1 computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 10–14. DOI : <https://doi.org/10.1109/ISSCC.2014.6757323>
- [13] Akhilesh Jaiswal, Indranil Chakraborty, Amogh Agrawal, and Kaushik Roy. 2019. 8T SRAM cell as a multibit dot-product engine for beyond von Neumann computing. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (2019), 2556–2567. DOI : <https://doi.org/10.1109/TVLSI.2019.2929245>
- [14] Hongyang Jia, Hossein Valavi, Yinqi Tang, Jintao Zhang, and Naveen Verma. 2020. A programmable heterogeneous microprocessor based on bit-scalable in-memory computing. *IEEE Journal of Solid-State Circuits* 55, 9 (2020), 2609–2621. DOI : <https://doi.org/10.1109/JSSC.2020.2987714>
- [15] Hongwu Jiang, Shanshi Huang, Wantong Li, and Shimeng Yu. 2023. ENNA: An efficient neural network accelerator design based on ADC-free compute-in-memory subarrays. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 1 (2023), 353–363. DOI : <https://doi.org/10.1109/TCSI.2022.3208755>
- [16] Hongwu Jiang, Wantong Li, Shanshi Huang, Stefan Cosemans, Francky Catthoor, and Shimeng Yu. 2022. Analog-to-digital converter design exploration for compute-in-memory accelerators. *IEEE Design & Test* 39, 2 (2022), 48–55. DOI : <https://doi.org/10.1109/MDAT.2021.3050715>
- [17] Hongwu Jiang, Xiaochen Peng, Shanshi Huang, and Shimeng Yu. 2020. CIMAT: A compute-in-memory architecture for on-chip training based on transpose SRAM arrays. *IEEE Transactions on Computers* 69, 7 (2020), 944–954. DOI : <https://doi.org/10.1109/TC.2020.2980533>
- [18] Honggu Kim, Yerim An, Ryunyeong Kim, Sunyoung Kim, and Yong Shim. 2024. Union SRAM: PVT variation auto-compensated, bit precision configurable current mode 8T SRAM in memory MAC macro. *IEEE Access* 12 (2024), 162882–162893. DOI : <https://doi.org/10.1109/ACCESS.2024.3487975>
- [19] Hyunjoon Kim, Taegun Yoo, Tony Tae-Hyoung Kim, and Bongjin Kim. 2021. Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks. *IEEE Journal of Solid-State Circuits* 56, 7 (2021), 2221–2233. DOI : <https://doi.org/10.1109/JSSC.2021.3061508>
- [20] Yulhwa Kim, Hyungjun Kim, and Jae-Joon Kim. 2022. Extreme partial-sum quantization for analog computing-in-memory neural network accelerators. *Journal on Emerging Technologies in Computing Systems* 18, 4 (Oct. 2022), Article 75, 19 pages. DOI : <https://doi.org/10.1145/3528104>

- [21] Maha Kooli, Antoine Heraud, Henri-Pierre Charles, Bastien Giraud, Roman Gauchi, Mona Ezzadeen, Kevin Mambu, Valentin Egloff, and Jean-Philippe Noel. 2022. Towards a truly integrated vector processing unit for memory-bound applications based on a cost-competitive computational SRAM design solution. *Journal on Emerging Technologies in Computing Systems* 18, 2 (April 2022), Article 40, 26 pages. DOI : <https://doi.org/10.1145/3485823>
- [22] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report 0. University of Toronto, Toronto, Ontario. Retrieved from <https://www.cs.toronto.edu/kriz/learning-features-2009-TR.pdf>
- [23] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, Vol. 25. Retrieved from [https://proceedings.neurips.cc/paper\\_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf)
- [24] Navjot Kukreja, Alena Shilova, Olivier Beaumont, Jan Huckelheim, Nicola Ferrier, Paul Hovland, and Gerard Gorman. 2019. Training on the edge: The why and the how. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 899–903. DOI : <https://doi.org/10.1109/IPDPSW.2019.00148>
- [25] Dinesh Kushwaha, Jaya Kumar Abotula, Rajat Kohli, Jwalant Mishra, Sudeb Dasgupta, and Anand Bulusu. 2024. Multi-bit compute-in memory architecture using a C-2C ladder network. *IEEE Transactions on Circuits and Systems II: Express Briefs* 71, 6 (2024), 3166–3170. DOI : <https://doi.org/10.1109/TCSII.2023.3329261>
- [26] Haitong Li, Mudit Bhargava, Paul N. Whatmough, and H.-S. Philip Wong. 2019. On-chip memory technology design space explorations for mobile deep neural network accelerators. In *56th Annual Design Automation Conference 2019 (DAC '19)*. ACM, New York, NY, Article 131, 6 pages. DOI : <https://doi.org/10.1145/3316781.3317874>
- [27] Ilya Loshchilov and Frank Hutter. 2017. SGDR: Stochastic gradient descent with warm restarts. arXiv:1608.03983. Retrieved from <https://arxiv.org/abs/1608.03983>
- [28] Mahta Mayahinia, Abhairaj Singh, Christopher Bengel, Stefan Wiefels, Muath A. Lebdeh, Stephan Menzel, Dirk J. Wouters, Anteneh Gebregiorgis, Rajendra Bishnoi, Rajiv Joshi, et al. 2022. A voltage-controlled, oscillation-based ADC design for computation-in-memory architectures using emerging ReRAMs. *Journal on Emerging Technologies in Computing Systems* 18, 2 (March 2022), Article 32, 25 pages. DOI : <https://doi.org/10.1145/3451212>
- [29] Sparsh Mittal, Gaurav Verma, Brajesh Kaushik, and Farooq A. Khanday. 2021. A survey of SRAM-based in-memory computing techniques and applications. *Journal of Systems Architecture* 119 (2021), 102276. DOI : <https://doi.org/10.1016/j.sysarc.2021.102276>
- [30] Ali Bou Nassif, Ismail Shahin, Imtinan Attili, Mohammad Azzeh, and Khaled Shaalan. 2019. Speech recognition using deep neural networks: A systematic review. *IEEE Access* 7 (2019), 19143–19165. DOI : <https://doi.org/10.1109/ACCESS.2019.2896880>
- [31] Duy-Thanh Nguyen, Abhiroop Bhattacharjee, Abhishek Moitra, and Priyadarshini Panda. 2024. MCAIMem: A mixed SRAM and eDRAM cell for area and energy-efficient on-chip AI memory. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32, 11 (2024), 2023–2036. DOI : <https://doi.org/10.1109/TVLSI.2024.3439231>
- [32] Leibin Ni, Hantao Huang, Zichuan Liu, Rajiv V. Joshi, and Hao Yu. 2017. Distributed in-memory computing on binary RRAM crossbar. *Journal on Emerging Technologies in Computing Systems* 13, 3, (March 2017), Article 36, 18 pages. DOI : <https://doi.org/10.1145/2996192>
- [33] Xiaochen Peng, Shanshi Huang, Hongwu Jiang, Anni Lu, and Shimeng Yu. 2021. DNN+NeuroSim V2.0: An end-to-end benchmarking framework for compute-in-memory accelerators for on-chip training. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 11 (2021), 2306–2319. DOI : <https://doi.org/10.1109/TCAD.2020.3043731>
- [34] Kartik Prabhu, Albert Gural, Zainab F. Khan, Robert M. Radway, Massimo Giordano, Kalhan Koul, Rohan Doshi, John W. Kustin, Timothy Liu, Gregorio B. Lopes, et al. 2022. CHIMERA: A 0.92-TOPS, 2.2-TOPS/W edge AI accelerator with 2-MByte on-chip foundry resistive RAM for efficient training and inference. *IEEE Journal of Solid-State Circuits* 57, 4 (2022), 1013–1026. DOI : <https://doi.org/10.1109/JSSC.2022.3140753>
- [35] Ximing Qiao, Xiong Cao, Huanrui Yang, Linghao Song, and Hai Li. 2018. Atomlayer: A universal reRAM-based CNN accelerator with atomic layer computation. In *55th Annual Design Automation Conference (DAC '18)*. ACM, New York, NY, Article 103, 6 pages. DOI : <https://doi.org/10.1145/3195970.3195998>
- [36] Shubham Sahay, Mohammad Bavandpour, Mohammad Reza Mahmoodi, and Dmitri Strukov. 2020. A 2T-1R cell array with high dynamic range for mismatch-robust and efficient neurocomputing. In *2020 IEEE International Memory Workshop (IMW)*, 1–4. DOI : <https://doi.org/10.1109/IMW48823.2020.9108142>
- [37] Abu Sebastian, Manuel Le Gallo, Riduan Khaddam-Aljameh, and Evangelos Eleftheriou. 2020. Memory devices and applications for in-memory computing. *Nature Nanotechnology* 15, 7 (2020), 529–544. DOI : <https://doi.org/10.1038/s41565-020-0655-z>

- [38] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 14–26. DOI: <https://doi.org/10.1109/ISCA.2016.12>
- [39] Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. arXiv:1409.1556. Retrieved from <https://arxiv.org/abs/1409.1556>
- [40] Raghav Singhal, Vivek Saraswat, Shreyas Deshmukh, Sreenivas Subramoney, Laxmeesha Somappa, Maryam Shojaei Baghini, and Udayan Ganguly. 2023. Enhanced regularization for on-chip training using analog and temporary memory weights. *Neural Networks* 165 (2023), 1050–1057. DOI: <https://doi.org/10.1016/j.neunet.2023.07.001>
- [41] Jian-Wei Su, Xin Si, Yen-Chi Chou, Ting-Wei Chang, Wei-Hsing Huang, Yung-Ning Tu, Ruhui Liu, Pei-Jung Lu, Ta-Wei Liu, Jing-Hong Wang, et al. 2022. Two-way transpose multibit 6T SRAM computing-in-memory macro for inference-training AI edge chips. *IEEE Journal of Solid-State Circuits* 57, 2 (2022), 609–624. DOI: <https://doi.org/10.1109/JSSC.2021.3108344>
- [42] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2020. How to evaluate deep neural network processors: TOPS/W (alone) considered harmful. *IEEE Solid-State Circuits Magazine* 12, 3 (2020), 28–41. DOI: <https://doi.org/10.1109/MSSC.2020.3002140>
- [43] Fengbin Tu, Weiwei Wu, Shouyi Yin, Leibo Liu, and Shaojun Wei. 2018. RANA: Towards efficient neural acceleration with refresh-optimized embedded DRAM. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 340–352. DOI: <https://doi.org/10.1109/ISCA.2018.00037>
- [44] Naveen Verma, Hongyang Jia, Hossein Valavi, Yinqi Tang, Murat Ozatay, Lung-Yen Chen, Bonan Zhang, and Peter Deaville. 2019. In-memory computing: Advances and prospects. *IEEE Solid-State Circuits Magazine* 11, 3 (2019), 43–55. DOI: <https://doi.org/10.1109/MSSC.2019.2922889>
- [45] Hechen Wang, Renzhi Liu, Richard Dorrance, Deepak Dasalukunte, Dan Lake, and Brent Carlton. 2023. A charge domain SRAM compute-in-memory macro with C-2C ladder-based 8-bit MAC unit in 22-nm FinFET process for edge inference. *IEEE Journal of Solid-State Circuits* 58, 4 (2023), 1037–1050. DOI: <https://doi.org/10.1109/JSSC.2022.3232601>
- [46] Bo Zhang, Shihui Yin, Minkyu Kim, Jyotishman Saikia, Soonwan Kwon, Sungmeen Myung, Hyunsoo Kim, Sang Joon Kim, Jae-Sun Seo, and Mingoo Seok. 2023. PIMCA: A programmable in-memory computing accelerator for energy-efficient DNN inference. *IEEE Journal of Solid-State Circuits* 58, 5 (2023), 1436–1449. DOI: <https://doi.org/10.1109/JSSC.2022.3211290>
- [47] Jintao Zhang, Zhuo Wang, and Naveen Verma. 2017. In-memory computation of a machine-learning classifier in a standard 6T SRAM array. *IEEE Journal of Solid-State Circuits* 52, 4 (2017), 915–924. DOI: <https://doi.org/10.1109/JSSC.2016.2642198>
- [48] Sai Qian Zhang, Thierry Tambe, Nestor Cuevas, Gu-Yeon Wei, and David Brooks. 2024. CAMEL: Co-designing AI models and eDRAMs for efficient on-device learning. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 861–875. DOI: <https://doi.org/10.1109/HPCA57654.2024.00071>

Received 7 May 2024; revised 7 April 2025; accepted 22 August 2025